SIT NAGPUR • Dept. of Computer Science • JavaScript Lab Tanmay Sankulwar | PRN: 24070521058

# EXPERIMENT 10

**Student Name:** Tanmay Sankulwar
**PRN:** 24070521058
**File Path:** `EXP10/Practice/`, `EXP10/JSON/`, `EXP10/Casestudy/`

## Experiment Title

**JSON Data Processing, Serialization, and Dynamic Asynchronous Loading using Fetch API & jQuery AJAX**

## Objective

1. To understand the structure and syntax of JavaScript Object Notation (JSON) as a universal, lightweight, language-independent data-interchange format.
2. To serialize in-memory JavaScript objects into formatted JSON strings using `JSON.stringify()` and parse JSON strings back into live objects using `JSON.parse()`.
3. To dynamically construct and manipulate HTML table rows from user input using standard DOM methods (`document.createElement`, `appendChild`, and `textContent`).
4. To fetch external JSON data asynchronously using the native JavaScript `fetch()` API and handle Promise resolution chains (`.then()` and `.catch()`).
5. To load and process JSON datasets using jQuery AJAX shorthand methods (`$.getJSON()`, `$.each()`), computing real-time aggregate dashboard metrics (`totalOrders` and `totalSales`).
6. To implement client-side persistence by pairing JSON serialization with `localStorage` and `sessionStorage` in event registration and ticket booking workflows.

## Software / Tools Required

1. **Operating System:** Windows 10 / 11
2. **Development Environment:** Visual Studio Code / Antigravity IDE
3. **Web Browser:** Google Chrome / Modern Web Browser
4. **Languages & Technologies:** HTML5, CSS3, Modern JavaScript (ES6+), jQuery 3.7.1, JSON (JavaScript Object Notation)

## Program / Task

### Task 10.a — Employee Details Management & JSON Serialization

**Files:** `EXP10/Practice/index.html`, `EXP10/Practice/script.js`, `EXP10/Practice/employees.json`, `EXP10/Practice/style.css`

The application provides a comprehensive Employee Management interface consisting of: - **Interactive Employee Form:** Collects `Employee ID`, `Employee Name`, `Department`, and `Salary`. Validates that no empty values are submitted. - **In-Memory Object Construction & JSON Formatting:** Constructs an employee object and serializes it using `JSON.stringify(employee, null, 4)` to produce human-readable indented JSON rendered live inside a `<pre id="jsonOutput">` preview block. - **Dynamic DOM Table Insertion:** Creates table rows (`tr`) with formatted cells (`td`) and prepends currency formatting (`₹`) before appending the record directly to the table body (`#employeeTableBody`). - **External Dataset Loading (`loadEmployees`):** Uses the `fetch("employees.json")` API to retrieve pre-configured employee records, validates HTTP response status (`response.ok`), iterates over each record using `forEach()`, and dynamically populates the tabular view.

### Task 10.b — Asynchronous Student Details Loading via Fetch API

**Files:** `EXP10/JSON/index.html`, `EXP10/JSON/students.json`

A focused asynchronous data fetching application featuring: - **Button-Triggered Data Retrieval:** An interactive **Load Students** button that triggers the `loadStudents()` function upon invocation. - **Promise-Based Pipeline:** Initiates `fetch('students.json')`, pipes the response through `response.json()`, clears the previous table body, and renders records (`ID`, `Name`, `Department`, `Marks`) with ES6 template literal row strings. - **Graceful Error Handling:** Implements `.catch(error => console.log('Error:', error))` to report any network or parsing failures without crashing the application.

## Source Code

### Main Task 10.a — Employee Details HTML ( `EXP10/Practice/index.html` )

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Employee Details using JSON</title>
    <link rel="stylesheet" href="style.css">
</head>

<body>

    <div class="container">

        <h1>Employee Details using JSON</h1>

        <div class="form-box">

            <h2>Enter Employee Details</h2>

            <label for="employeeId">Employee ID</label>
            <input type="text" id="employeeId" placeholder="Enter Employee ID">

            <label for="employeeName">Employee Name</label>
            <input type="text" id="employeeName" placeholder="Enter Employee Name">

            <label for="department">Department</label>
            <input type="text" id="department" placeholder="Enter Department">

            <label for="salary">Salary</label>
            <input type="number" id="salary" placeholder="Enter Salary">

            <button onclick="addEmployee()">Add Employee</button>

        </div>

        <div class="json-box">
            <h2>Employee JSON Data</h2>
            <pre id="jsonOutput"></pre>
        </div>

        <div class="table-box">

            <h2>Employee Information</h2>

            <button onclick="loadEmployees()">Load Employees</button>

            <table>
                <thead>
                    <tr>
                        <th>Employee ID</th>
                        <th>Employee Name</th>
                        <th>Department</th>
                        <th>Salary</th>
                    </tr>
                </thead>

                <tbody id="employeeTableBody">
                </tbody>
            </table>
```

```
        </div>

    </div>

    <script src="script.js"></script>

</body>
</html>
```

## Main Task 10.a — Employee Details JavaScript ( `EXP10/Practice/script.js` )

```
function addEmployee() {

    let employeeId = document.getElementById("employeeId").value;
    let employeeName = document.getElementById("employeeName").value;
    let department = document.getElementById("department").value;
    let salary = document.getElementById("salary").value;

    if (employeeId === "" || employeeName === "" || department === "" || salary === "") {
        alert("Please enter all employee details.");
        return;
    }

    let employee = {
        employeeId: employeeId,
        employeeName: employeeName,
        department: department,
        salary: salary
    };

    let jsonData = JSON.stringify(employee, null, 4);

    document.getElementById("jsonOutput").textContent = jsonData;

    let tableBody = document.getElementById("employeeTableBody");

    let row = document.createElement("tr");

    row.innerHTML = `
        <td>${employee.employeeId}</td>
        <td>${employee.employeeName}</td>
        <td>${employee.department}</td>
        <td>₹${employee.salary}</td>
    `;

    tableBody.appendChild(row);

    document.getElementById("employeeId").value = "";
    document.getElementById("employeeName").value = "";
    document.getElementById("department").value = "";
    document.getElementById("salary").value = "";
}


function loadEmployees() {

    fetch("employees.json")
        .then(response => {
            if (!response.ok) {
                throw new Error("Unable to load employees.json");
            }

            return response.json();
        })

        .then(data => {

            let tableBody = document.getElementById("employeeTableBody");

            tableBody.innerHTML = "";

            data.forEach(employee => {
```

```
                let row = document.createElement("tr");

                row.innerHTML = `
                    <td>${employee.employeeId}</td>
                    <td>${employee.employeeName}</td>
                    <td>${employee.department}</td>
                    <td>₹${employee.salary}</td>
                `;

                tableBody.appendChild(row);
            });

        })

        .catch(error => {
            console.error(error);
            alert("Error loading employee data. Run the project using Live Server.");
        });
}
```

## Main Task 10.b — Student Details using Fetch ( `EXP10/JSON/index.html` )

```
<!DOCTYPE html>
<html>
    <head>
        <title>Load JSON using Fetch</title>
        <style>
            table {
                width: 70%;
                border-collapse: collapse;
                margin-top: 20px;
            }
            th, td {
                border: 1px solid black;
                padding: 10px;
                text-align: left;
            }
            th{
                background-color: #f2f2f2;
            }
        </style>
    </head>
    <body>
        <h2>Student Details using fetch()</h2>
        <button onclick="loadStudents()">Load Students</button>
        <table id="studentTable">
            <thead>
                <tr>
                    <th>ID</th>
                    <th>Name</th>
                    <th>Department</th>
                    <th>Marks</th>
                </tr>
            </thead>
            <tbody>
            </tbody>
        </table>

<script>
    function loadStudents() {
        fetch('students.json')
            .then(response => response.json())
            .then(data => {
                let tableBody = document.querySelector('#studentTable tbody');
                tableBody.innerHTML = '';
                data.forEach(student => {
                    let row = `
                    <tr>
                        <td>${student.id}</td>
                        <td>${student.name}</td>
                        <td>${student.department}</td>
                        <td>${student.marks}</td>
                    </tr>`;
                    tableBody.innerHTML += row;
                });
            })
            .catch(error => {
                console.log('Error:', error)
                });
    }
</script>
</body>
</html>
```

## Explanation / Concepts

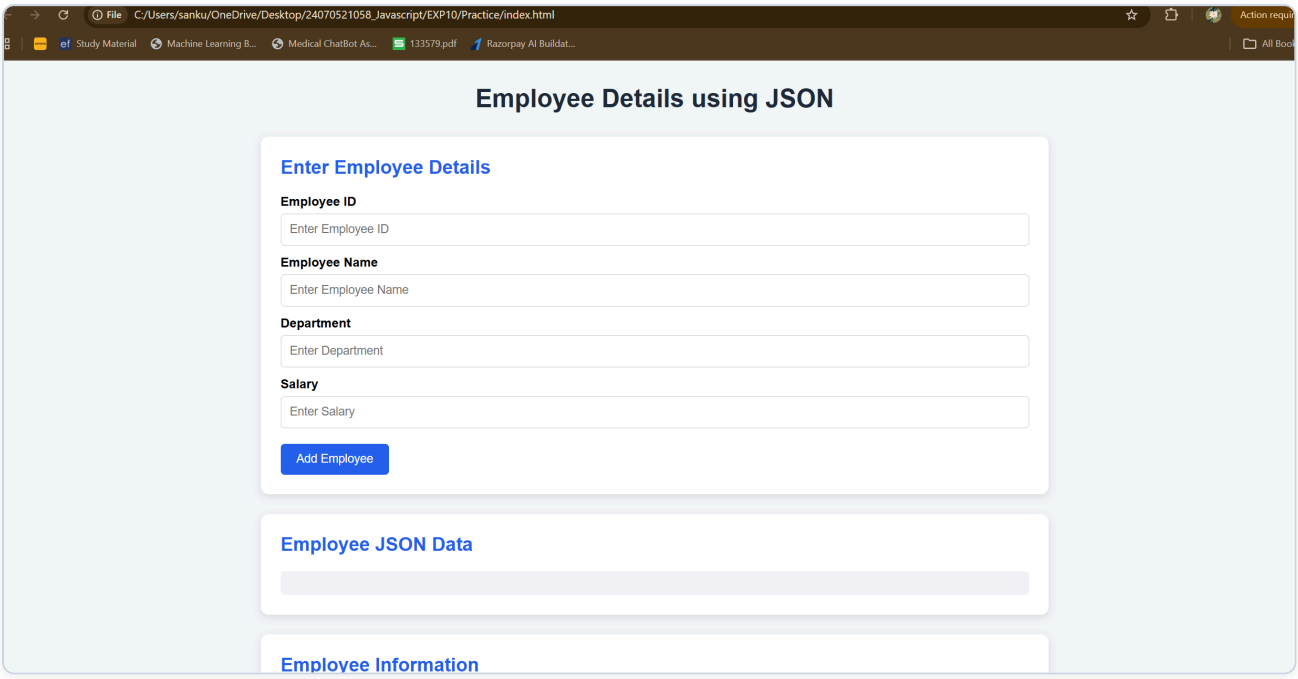
1. **JSON Syntax & Structure:**
2. JSON keys must be string literals enclosed in double quotes ( `"key": "value"` ).
3. Supported data types include strings, numbers, booleans, arrays, nested objects, and `null` . Functions, undefined, and symbols are intentionally excluded.
4. `JSON.stringify(value, replacer, space)` **:**
5. Serializes a JavaScript runtime object into an equivalent JSON-encoded text string.
6. Passing an integer (e.g. `4` ) as the third parameter introduces uniform indentation and newline formatting, essential for readability in debug blocks and `<pre>` elements.
7. **Promise-Based Asynchronous** `fetch()` **API:**
8. Native modern browser API replacing legacy `XMLHttpRequest` .
9. `fetch()` returns a `Promise` that resolves to the `Response` object representing the HTTP response.
10. Calling `response.json()` parses the response stream as JSON, returning a second Promise containing the evaluated JavaScript object/array.
11. **DOM Node Creation vs. InnerHTML:**
12. Using `document.createElement("tr")` combined with scoped innerHTML templating enables structured, safe dynamic list generation without resetting existing child listeners.
13. **Cross-Origin & Protocol Security:**
14. Browsers enforce CORS and file security rules preventing direct `file://` protocol fetching in some contexts. Launching applications through a local HTTP development server (e.g., Live Server) ensures clean Promise fulfillment.

## Output

1. **Employee Management Portal:**
2. Inputting `Employee ID: E101` , `Employee Name: Sanjay` , `Department: CSE` , and `Salary: 45000` serializes the record into an indented JSON preview block and appends a row into the dynamic table.
3. Clicking **Load Employees** queries `employees.json` asynchronously, populating records for all department employees ( `Sanjay` , `Tanmay` , `Samruddhi` , `Atharv` ).
4. **Student Details Fetch:**
5. Clicking **Load Students** invokes `students.json` via `fetch()` , clearing the previous body and generating rows displaying student IDs, names, departments, and marks.

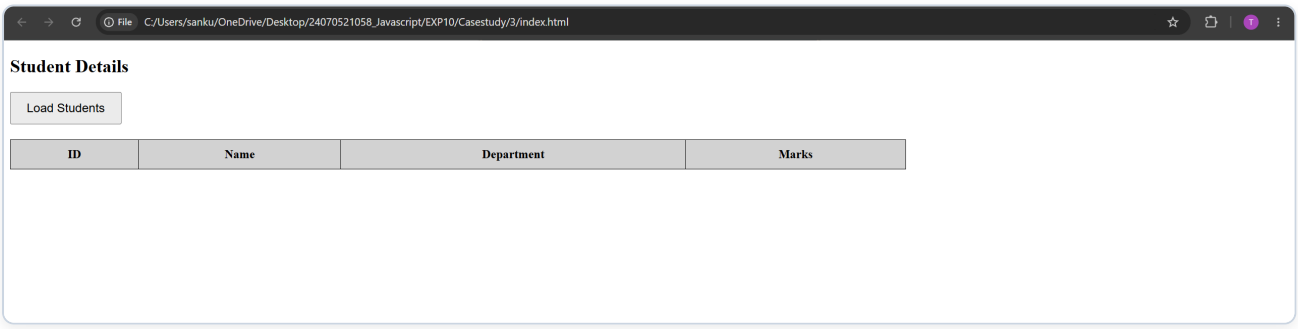
## Screenshots

### Employee Details using JSON Output




*Screenshot 2026-10-21 225208.png*

### Student Details using Fetch API Output




*Screenshot 2026-10-21 225452.png*

## Case Study

### Case Study 1 — Online Food Order Dashboard with jQuery AJAX

**Files:** `EXP10/Casestudy/4(JSON/index.html`, `EXP10/Casestudy/4(JSON/ORDERS.JSON`

The dashboard implements an enterprise food order monitoring system utilizing jQuery AJAX for asynchronous data retrieval and real-time financial aggregation.

**Key Features:**

- **jQuery** `$.getJSON()` **Pipeline:** Consumes `ORDERS.JSON` asynchronously when clicking the **View Orders** button.

- **Dynamic Table Generation:** Uses `$.each(data, function(index, order) { ... })` to append structured table rows representing individual customer orders (`orderId`, `customerName`, `foodItem`, `quantity`, `totalAmount`, `orderStatus`).
- **Real-Time Financial Metrics:** Simultaneously calculates the total number of orders and total revenue using `parseFloat(order.totalAmount).toFixed(2)`, dynamically populating the summary banner cards (`totalOrders` and `totalSales`).
- **Network Resilience:** Implements `.done()` and `.fail()` callbacks; on network interruption, the interface safely resets counters and displays an inline warning banner.

```
// Excerpt from EXP10/Casestudy/4(JSON/index.html
$("#viewOrders").click(function() {
    $("#orderBody").empty();
    $("#message").text("");

    $.getJSON("orders.json")
        .done(function(data) {
            let totalOrders = 0;
            let totalSales = 0;

            $.each(data, function(index, order) {
                let row = `
                    <tr>
                        <td>${order.orderId}</td>
                        <td>${order.customerName}</td>
                        <td>${order.foodItem}</td>
                        <td>${order.quantity}</td>
                        <td>₹${parseFloat(order.totalAmount).toFixed(2)}</td>
                        <td>${order.orderStatus}</td>
                    </tr>
                `;
                $("#orderBody").append(row);
                totalOrders++;
                totalSales += parseFloat(order.totalAmount);
            });

            $("#totalOrders").text(totalOrders);
            $("#totalSales").text(totalSales.toFixed(2));
        })
        .fail(function() {
            $("#orderBody").empty();
            $("#totalOrders").text("0");
            $("#totalSales").text("0.00");
            $("#message").text("Unable to load orders. Please make sure orders.json is available.");
        });
});
```

### Online Food Order Dashboard Screenshot




---

### Case Study 2 — Technical Workshop Ticket Booking with Web Storage & JSON Persistence

**Files:** `EXP10/Casestudy/1/index.html`, `EXP10/Casestudy/1/script.js`, `EXP10/Casestudy/1/style.css`

The case study provides a complete workshop registration management system integrating multi-input validation and client-side data serialization: - **Real-Time Input Validation:** Attaches `focus`, `blur`, and `input`

events across inputs; applies visual cues (green for valid, red for invalid, regex sanitization for mobile digits). - **Dual-Storage Persistence:** Implements **Save Local** ( `localStorage.setItem("workshopData", JSON.stringify(data))` ) and **Save Session** ( `sessionStorage.setItem(...)` ), facilitating both permanent device storage and session-scoped backups. - **State Rehydration:** The **Load Data** button deserializes the saved JSON string with `JSON.parse()`, repopulating form controls, radio buttons, and checkboxes automatically.

---

## Result / Conclusion

Experiment 10 was successfully implemented and verified. The practical demonstrated the core capabilities of JSON in modern JavaScript applications: converting runtime structures into standardized interchange strings with `JSON.stringify()`, deserializing payloads with `JSON.parse()`, loading remote datasets with the native `fetch()` Promise pipeline, and building interactive, aggregate-computing dashboards using jQuery AJAX.